Then you can send a request to the application to fetch all the books in the catalog:

```
$ http :9001/books
```

It should return the test data we created in listing 4.6. When you're done, stop the application with Ctrl-C.

NOTE Instead of using profiles as feature flags, a more scalable and structured approach is defining custom properties to configure functionality, and relying on annotations such as `@ConditionalOnProperty` and `@ConditionalOnCloudPlatform` to control when certain beans should be loaded into the Spring application context. That's one of the foundations of Spring Boot auto-configuration. For example, you could define a `polar.testdata.enabled` custom property and use the `@ConditionalOnProperty(name = "polar.testdata.enabled", havingValue = "true")` annotation on the `BookDataLoader` class.

Next, I'll show you how to use profiles to group configuration data.

USING PROFILES AS CONFIGURATION GROUPS

The Spring Framework's profile functionality allows you to register some beans only if a given profile is active. Likewise, Spring Boot lets you define configuration data that is loaded only when a specific profile is active. A common way to do that is inside a property file named with the profile as a suffix. In the case of the Catalog Service, you could create a new application-dev.yml file and define a value for the `polar.greeting` property, which would be used by Spring Boot only if the `dev` profile was active. Profile-specific property files take precedence over the non-specific property files, so the value defined in application-dev.yml would take precedence over the one in application.yml.